



GIL

Accounting Platform / Nuxt + PostgreSQL

GIL adalah platform akuntansi enterprise yang dirancang sebagai alternatif modern dari Accurate, mengintegrasikan modul inti akuntansi dengan fitur baru khas enterprise: **budgeting per departemen**, **pemantauan transaksi real-time**, **close/open period**, dan **role-based access control** yang granular. Dibangun di atas stack **Nuxt 3** dan **PostgreSQL** untuk performa, keamanan, dan skalabilitas.

VERSI DOKUMEN
1.0

TIPE
Technical Design

STACK
Nuxt 3 · PG 16

NUXT 3

POSTGRESQL

RBAC

BUDGETING

Daftar Isi

1 Pendahuluan

2 Visi, Misi & Tujuan Produk

3 Arsitektur Sistem

4 Modul Aplikasi

- 4.1 General Ledger
- 4.2 Accounts Receivable (Piutang)
- 4.3 Accounts Payable (Hutang)
- 4.4 Penjualan (Sales)
- 4.5 Pembelian (Purchase)
- 4.6 Persediaan (Inventory)
- 4.7 Manufaktur
- 4.8 Aset Tetap
- 4.9 Kas & Bank

5 Budgeting per Departemen & Budget Component

6 Pemantauan Transaksi

7 Close/Open Period

8 Role-Based Access Control (RBAC)

9 Model Data & Skema Database

10 Tech Stack & Konfigurasi

11 Roadmap Implementasi

12 Penutup

SECTION 1

Pendahuluan

Latar Belakang

Aplikasi akuntansi merupakan tulang punggung operasional keuangan bagi setiap organisasi, mulai dari usaha kecil hingga korporasi berskala enterprise. Di Indonesia, Accurate telah menjadi salah satu pemain utama selama lebih dari dua dekade, menyediakan modul-modul inti seperti buku besar, piutang, hutang, persediaan, hingga manufaktur. Namun seiring perkembangan kebutuhan bisnis modern, terdapat celah fungsional yang belum sepenuhnya terpenuhi oleh solusi existing, khususnya pada aspek **budgeting terdistribusi per departemen**, **pemantauan transaksi real-time**, kontrol periode akuntansi yang ketat, serta granularitas hak akses berbasis peran (RBAC).

GIL hadir sebagai jawaban atas kebutuhan tersebut. Nama GIL diambil dari mata uang dalam semesta game Final Fantasy — sebuah mata uang universal yang melintasi berbagai kerajaan dan era. Filosofi nama ini melambangkan ambisi GIL untuk menjadi platform akuntansi yang universal, lintas-departemen, lintas-periode, dan lintas-role, dengan tetap mempertahankan kemudahan penggunaan yang menjadi daya tarik utama kelas aplikasi sejenis. GIL bukan sekadar replikasi Accurate, melainkan evolusi yang menggabungkan kekuatan modul akuntansi klasik dengan fitur enterprise modern.

Dari sisi teknologi, GIL dibangun di atas stack modern: **Nuxt 3** sebagai fullstack framework (frontend SSR/SPA + backend API dalam satu codebase), **PostgreSQL 16** sebagai database relasional untuk konsistensi transaksional ACID, serta ekosistem pendukung seperti Prisma/Drizzle ORM, Tailwind CSS, dan Docker. Pilihan stack ini menjamin produktivitas pengembangan, performa tinggi, dan kemudahan deployment baik on-premise maupun cloud.

Ruang Lingkup

Dokumen ini merupakan **design document** komprehensif yang mencakup arsitektur sistem, detail modul aplikasi, skema database, mekanisme RBAC, tech stack, hingga roadmap implementasi GIL. Dokumen ini menjadi rujukan utama bagi tim engineering, product, dan stakeholder selama fase pengembangan. Dokumen tidak mencakup detail implementasi kode per baris, melainkan berfokus pada keputusan desain, struktur data, dan kontrak antar modul.

- **Modul akuntansi inti:** General Ledger, AR, AP, Penjualan, Pembelian, Persediaan, Manufaktur, Aset Tetap, Kas & Bank.
- **Modul baru khas GIL:** Budgeting per Departemen & Budget Component, Pemantauan Transaksi, Close/Open Period, RBAC granular.
- **Layer teknis:** arsitektur Nuxt 3, skema PostgreSQL, mekanisme autentikasi & otorisasi, deployment topology.
- **Layer operasional:** audit trail, logging, monitoring, backup-restore strategy.

- **Di luar lingkup:** detail UI/UX per layar, materi training end-user, kontrak SLA, skema harga komersial.

| Target Pengguna

GIL ditujukan untuk kelas menengah ke atas (mid-to-large enterprise) dengan kompleksitas operasional yang membutuhkan kontrol anggaran terdistribusi dan audit transaksi yang ketat. Segmen target utama mencakup perusahaan manufaktur, distribusi, jasa profesional multi-cabang, koperasi besar, BUMDes, hingga organisasi non-profit yang memiliki struktur multi-departemen. Aplikasi ini juga relevan bagi holding company yang mengelola beberapa entitas bisnis dengan kebijakan akuntansi terpusat namun operasional terdesentralisasi.

Pengguna akhir GIL terdiri dari beberapa persona: **Super Admin** yang mengelola konfigurasi sistem dan RBAC, **Finance Manager** yang mengawasi siklus akuntansi dan close period, **Accountant** yang melakukan pencatatan transaksi harian, **Budget Owner** di setiap departemen yang mengelola anggaran dan approval realisasi, **Department Head** yang memantau kinerja keuangannya, serta **Auditor** internal/eksternal dengan akses read-only penuh pada data transaksi.

SECTION 2

Visi, Misi & Tujuan Produk

Visi

Menjadi **platform akuntansi enterprise terkemuka di Indonesia** yang menyatukan modul inti akuntansi klasik dengan kontrol anggaran terdistribusi, visibilitas transaksi real-time, dan tata kelola akses berbasis peran — semuanya dalam satu codebase modern berbasis Nuxt dan PostgreSQL.

Misi

- Menyediakan modul akuntansi lengkap yang setara dengan aplikasi kelas Accurate, dengan UX yang lebih modern.
- Memberikan kontrol anggaran terdistribusi melalui budgeting per departemen dan budget component yang dapat dikustomisasi.
- Menghadirkan visibilitas penuh terhadap transaksi melalui dashboard pemantauan real-time dan audit trail komprehensif.
- Memastikan integritas periode akuntansi dengan mekanisme close/open period yang ketat namun fleksibel.
- Menerapkan RBAC granular sehingga setiap aksi sensitif dapat dibatasi berdasarkan peran, departemen, dan modul.
- Membangun di atas stack open-source modern untuk menghindari vendor lock-in dan memudahkan kustomisasi.

Prinsip Desain

Prinsip	Penjelasan
Auditability First	Setiap transaksi harus terlacak: siapa, kapan, apa, dan mengapa. Audit trail bukan fitur tambahan, melainkan default.
Period Integrity	Integritas periode akuntansi tidak boleh dilanggar. Transaksi pada periode tertutup harus ditolak kecuali melalui mekanisme reopen yang teraudit.
Budget Discipline	Setiap pengeluaran terikat pada budget component. Overbudget harus melalui approval khusus, tidak boleh diam-diam.
Least Privilege	RBAC mengikuti prinsip least privilege. Role default adalah "no access"; hak akses diberikan secara eksplisit.
Double-Entry Immutable	Jurnal akuntansi double-entry bersifat immutable setelah diposting. Koreksi dilakukan melalui jurnal penyesuaian, bukan edit langsung.
API-First	Setiap fitur dapat diakses via API terstruktur. UI hanyalah salah satu konsumen API, bukan satu-satunya pintu masuk.
Multi-Tenant Ready	Skema database dirancang agar dapat di-extend ke multi-tenant tanpa refactor besar.

Tujuan & Indikator Kesuksesan

Tujuan jangka pendek GIL (12 bulan pertama) adalah merilis MVP yang mencakup modul inti akuntansi (GL, AR, AP), RBAC, dan modul budgeting. Indikator kesuksesan MVP meliputi: (1) onboarding minimal 3 pilot customer dalam 6 bulan setelah rilis, (2) zero data corruption pada transaksi double-entry, (3) waktu respons API rata-rata di bawah 300ms untuk operasi CRUD standar, dan (4) 95% test coverage pada modul budgeting dan GL.

Tujuan jangka panjang (24-36 bulan) adalah menjadi platform akuntansi pilihan untuk segmen mid-market di Indonesia, dengan minimal 50 tenant aktif dan ekosistem integrasi (e-Faktur, e-Bupot, marketplace, payment gateway). Indikatornya mencakup Net Revenue Retention di atas 110%, jumlah modul terintegrasi di atas 20, serta sertifikasi kepatuhan terhadap standar akuntansi Indonesia (PSAK) dan regulasi perpajakan terbaru.

SECTION 3

Arsitektur Sistem

Gambaran Umum

GIL mengadopsi arsitektur **monolitik modular** dengan Nuxt 3 sebagai backbone. Pilihan monolitik modular (bukan microservices) dibuat dengan kesadaran penuh: pada fase awal, kompleksitas operasional microservices tidak sepadan dengan benefitnya. Nuxt 3 sendiri sudah menyediakan pemisahan jelas antara frontend (Vue 3 + SSR), backend (Nitro server routes), dan shared logic (composables, types), sehingga aplikasi tetap rapi tanpa overhead komunikasi antar-service. Migrasi ke microservices dapat dilakukan kemudian jika beban operasional menghendaki, dengan mengekstrak modul yang bottleneck menjadi service terpisah.

Layered Architecture

Layer	Komponen	Tanggung Jawab
Presentation	Vue 3 pages, components, layouts (Nuxt)	Rendering UI, form handling, client-side validation, route guard berbasis RBAC.
API Gateway	Nuxt server routes (/server/api)	Endpoint REST, autentikasi (JWT/session), otorisasi (RBAC middleware), rate limiting.
Application Service	Service modules (e.g. BudgetService, JournalService)	Orkestrasi use case, validasi business rule, koordinasi antar domain, transaction boundary.
Domain	Entity, value object, domain event	Logika bisnis murni: perhitungan jurnal, rule overbudget, state machine periode.
Infrastructure	Prisma/Drizzle ORM, Redis cache, mailer, file storage	Akses ke DB, cache, external service. Tidak mengandung business rule.
Database	PostgreSQL 16	Persistence, ACID, trigger audit, constraint, view, materialized view untuk reporting.

Deployment Topology

GIL direkomendasikan di-deploy menggunakan Docker Compose untuk instalasi on-premise, atau Kubernetes untuk skala cloud. Komponen minimum yang berjalan: (1) container aplikasi Nuxt (Nitro server), (2) container PostgreSQL 16, (3) container Redis untuk cache & session store, dan (4) reverse proxy (Nginx/Caddy) untuk TLS termination dan static asset serving. Untuk skala lebih besar, Nuxt app dapat di-scale horizontal di belakang load balancer, dengan PostgreSQL menggunakan read replica untuk query reporting.

Security Boundaries

Boundary keamanan GIL terdiri dari tiga lapis. Lapis pertama adalah **edge security**: TLS, CORS whitelist, rate limiting per IP, dan WAF opsional. Lapis kedua adalah **application security**: autentikasi berbasis session cookie httpOnly + CSRF token, otorisasi RBAC pada setiap endpoint API, serta input validation via Zod schema. Lapis ketiga adalah **data security**: enkripsi at-rest untuk kolom sensitif (PII, kredensial), audit trigger pada PostgreSQL untuk tabel kritis, dan

backup terenkripsi dengan retention policy.

SECTION 4

Modul Aplikasi

GIL mengadopsi struktur modul yang setara dengan aplikasi akuntansi kelas Accurate, dilengkapi empat modul baru khas GIL yang dibahas mendalam pada bab tersendiri (Bab 5-8). Bab ini berfokus pada modul inti akuntansi. Setiap modul memiliki kontrak yang jelas: **entitas** (master data), **transaksi** (dokumen yang menggerakkan angka), **laporan** (output analitis), dan **integrasi** (interaksi dengan modul lain).

Modul	Status	Fungsi Inti
General Ledger	Inti	Buku besar, jurnal, laporan keuangan
Accounts Receivable	Inti	Faktur penjualan kredit, penerimaan, aging piutang
Accounts Payable	Inti	Faktur pembelian kredit, pembayaran, aging hutang
Penjualan	Inti	Quotation, sales order, delivery, faktur
Pembelian	Inti	Purchase request, PO, penerimaan barang, faktur
Persediaan	Inti	Item, gudang, kartu stok, stock opname, BOM
Manufaktur	Inti	Work order, pengeluaran bahan, hasil produksi
Aset Tetap	Inti	Akuisisi, penyusutan, disposal
Kas & Bank	Inti	Buku kas, rekonsiliasi bank, transfer antar akun
Budgeting per Departemen	Baru	Anggaran per departemen & budget component
Pemantauan Transaksi	Baru	Dashboard real-time, audit trail, anomaly detection
Close/Open Period	Baru	Kontrol periode akuntansi, locking transaksi
RBAC	Baru	Manajemen user, role, permission, matrix akses

General Ledger (Buku Besar)

Modul **General Ledger** adalah jantung akuntansi GIL. Setiap transaksi dari modul lain (penjualan, pembelian, kas, dll.) pada akhirnya akan memposting jurnal ke GL. GL menyimpan **chart of account** (bagan akun) yang fleksibel dengan dukungan multi-level (akun, sub-akun, departemen), serta mendukung multi-currency dengan kurs otomatis maupun manual per transaksi. Setiap jurnal bersifat double-entry dan immutable setelah diposting; koreksi dilakukan melalui jurnal penyesuaian (reversing entry) yang juga tercatat di audit trail.

- **Entitas utama:** Account (no akun, nama, tipe, parent, departemen, mata uang), Journal (no, tanggal, periode, deskripsi), JournalLine (akun, debit, kredit, departemen, project).
- **Fitur kunci:** posting jurnal, reversing jurnal, jurnal penyesuaian, jurnal penutup akhir periode, recurring jurnal, multi-currency conversion.
- **Laporan:** Neraca, Laba Rugi, Arus Kas, Ekuitas, Trial Balance, General Ledger Detail, Work Sheet.

- **Integrasi:** setiap modul lain menjadi "source" jurnal otomatis. Budget component terhubung ke akun untuk kontrol realisasi.

Accounts Receivable (Piutang)

Modul **Accounts Receivable** mengelola siklus piutang dari pencatatan faktur penjualan kredit hingga pelunasan. Setiap pelanggan memiliki master data dengan limit kredit, term pembayaran, dan akun piutang default. Penerimaan pembayaran dapat berupa cash, transfer, cek, atau giro, dan mendukung alokasi sebagian (partial payment) maupun alokasi ke multiple faktur. Aging piutang dihitung otomatis berdasarkan tanggal jatuh tempo, dengan opsi notifikasi otomatis untuk faktur jatuh tempo.

- **Entitas:** Customer (master), Invoice (faktur penjualan kredit), Receipt (penerimaan pembayaran), Credit Note (retur/potongan).
- **Fitur:** limit kredit otomatis, term pembayaran per pelanggan, dunning letter, statement pelanggan bulanan, aging report (30/60/90/120).
- **Integrasi:** terhubung ke modul Penjualan (Sales Invoice → AR Invoice), Kas & Bank (Receipt), dan GL (auto-posting).

Accounts Payable (Hutang)

Modul **Accounts Payable** adalah cerminan AR, mengelola hutang ke pemasok. Pencatatan faktur pembelian kredit, retur, hingga pembayaran hutang terintegrasi dengan modul Pembelian dan Kas & Bank. Fitur approval workflow memungkinkan faktur hutang besar melewati persetujuan manager sebelum diposting. Aging hutang membantu tim finance memprioritaskan pembayaran berdasarkan jatuh tempo dan ketersediaan kas.

Penjualan (Sales)

Modul **Penjualan** mengelola siklus penjualan end-to-end: quotation, sales order, delivery order, sales invoice, hingga sales return. Setiap dokumen memiliki numbering sequence yang dapat dikustomisasi per cabang/departemen. Modul ini terintegrasi dengan modul Persediaan (auto-deplete stok saat delivery) dan AR (auto-create AR invoice saat sales invoice kredit).

- **Siklus:** Quotation → Sales Order → Delivery Order → Sales Invoice → (AR Invoice jika kredit) → Receipt.
- **Fitur:** pricing per pelanggan, discount multi-level, multi-currency, batch delivery, backorder handling.
- **Field kunci:** customer, salesperson, tanggal, term, item & qty, harga, diskon, pajak, total, akun pendapatan, akun piutang, departemen, project.

Pembelian (Purchase)

Modul **Pembelian** mengelola siklus pembelian: purchase request, purchase order, goods received note, purchase invoice, hingga purchase return. Sama seperti Penjualan, modul ini terintegrasi dengan Persediaan (auto-increase stok saat goods received), AP (auto-create AP invoice), dan Budgeting (validasi budget component saat PO dibuat).

Integrasi dengan modul Budgeting adalah salah satu diferensiator utama GIL. Saat sebuah Purchase Order dibuat, sistem akan memeriksa apakah nilai PO masih dalam sisa anggaran budget component terkait. Jika overbudget, PO memerlukan approval khusus dari Budget Owner level lebih tinggi atau Finance Manager. Hal ini mencegah pembelian impulsif yang melampaui anggaran departemen.

Persediaan (Inventory)

Modul **Persediaan** mengelola stok barang dengan dukungan multi-gudang, multi-unit (satuan), dan tiga metode evaluasi stok: FIFO, Moving Average, dan LIFO. Setiap pergerakan stok (masuk, keluar, transfer, opname) tercatat di kartu stok (stock card) yang menjadi sumber kebenaran history persediaan. Modul juga mendukung Bill of Material (BOM) untuk kebutuhan manufaktur.

- **Entitas:** Item (master, tipe: inventory, non-inventory, service, assembly), Warehouse, StockCard, StockAdjustment, BOM.
- **Fitur:** stock opname, transfer gudang, minimum/maximum stock alert, expired date tracking (untuk FMCG), serial number & batch tracking.
- **Laporan:** kartu stok, saldo stok per gudang, nilai persediaan, slow-moving inventory, stock aging.

Manufaktur

Modul **Manufaktur** mengelola proses produksi berbasis Bill of Material. Work Order dibuat berdasarkan BOM, kemudian bahan baku dikeluarkan dari gudang dan hasil produksi dimasukkan kembali sebagai item jadi. Modul juga mendukung pencatatan scrap (limbah produksi) dan downtime mesin. Biaya overhead produksi dapat dialokasikan ke hasil produksi berdasarkan rule yang dikonfigurasi.

Aset Tetap

Modul **Aset Tetap** mengelola siklus hidup aset dari akuisisi, penyusutan, hingga disposal. Tiga metode penyusutan didukung: garis lurus (straight-line), saldo menurun (declining balance), dan jumlah angka tahun (sum-of-years-digits). Setiap aset terhubung ke akun aset, akun akumulasi penyusutan, dan akun beban penyusutan di GL. Jurnal penyusutan dapat dijalankan otomatis tiap bulan.

Kas & Bank

Modul **Kas & Bank** mengelola transaksi tunai dan bank: penerimaan, pengeluaran, transfer antar akun, hingga rekonsiliasi bank. Modul mendukung pencatatan cek dan giros (baik keluar maupun masuk), termasuk tracking status cek/giro (di-pocket, ditolak, dicairkan). Rekonsiliasi bank memungkinkan matching antara pencatatan internal dengan rekening koran bank untuk mendeteksi discrepancy.

SECTION 5

Budgeting per Departemen & Budget Component

Konsep Inti

Modul **Budgeting** adalah salah satu diferensiator utama GIL. Modul ini memungkinkan organisasi membagi anggaran ke dalam **departemen** dan **budget component** — kategori anggaran yang dapat dikustomisasi (misalnya: gaji, travel, marketing, ATK, training, CAPEX IT). Setiap kombinasi (departemen × budget component × periode) memiliki nilai anggaran terdefinisi, dan setiap transaksi yang menggerakkan beban akan mengurangi sisa anggaran secara real-time.

Struktur Entitas

Entitas	Field Kunci	Deskripsi
Department	id, code, name, parent_id, head_user_id	Hierarki departemen; parent mendukung tree.
BudgetComponent	id, code, name, type (capex/opex), account_id	Kategori anggaran; terikat ke akun GL.
Budget	id, department_id, component_id, period_id, amount, status	Anggaran untuk kombinasi dept × component × period.
BudgetLine	id, budget_id, transaction_line_id, amount, type	Realisasi anggaran (encumbrance/actual).
BudgetTransfer	id, from_budget, to_budget, amount, approver	Transfer anggaran antar departemen/komponen.
BudgetRevision	id, budget_id, old_amount, new_amount, reason	Revisi anggaran dengan audit trail.

Siklus Anggaran

Siklus anggaran GIL mengikuti alur formal yang menjamin kontrol dan accountability. Tahap **Draft**: budget owner mengajukan nilai anggaran awal per komponen. Tahap **Review**: finance manager melakukan review dan negosiasi. Tahap **Approved**: anggaran disetujui dan siap digunakan pada periode berjalan. Tahap **Active**: anggaran aktif, transaksi mulai menggerakkan realisasi. Tahap **Closed**: periode berakhir, anggaran dikunci, variance dihitung untuk pelaporan.

State	Trigger	Aksi yang Diizinkan	Aktor
Draft	Budget owner membuat draft	Edit nilai, hapus, submit ke review	Budget Owner
Review	Submit dari Draft	Approve, reject, return to draft, nego nilai	Finance Manager
Approved	Approve dari Review	Aktivasi ke periode, revisi dengan approval	Finance Manager

State	Trigger	Aksi yang Diizinkan	Aktor
Active	Mulai periode	Transaksi realisasi, transfer antar komponen	All users (RBAC)
Closed	Akhir periode + close period	View-only, laporan variance, analisis post-mortem	All users (read-only)

Kontrol Overbudget

Ketika sebuah transaksi (PO, pengeluaran kas, jurnal beban) akan diposting, sistem melakukan **budget check** secara real-time: mencari Budget yang cocok (department × component × period), menghitung sisa (amount – encumbrance – actual), dan membandingkan dengan nilai transaksi. Jika transaksi melebihi sisa, sistem mengikuti policy yang dikonfigurasi:

- **Block (default):** transaksi ditolak dengan pesan "Overbudget — sisa anggaran Rp X".
- **Warn + Allow:** transaksi diizinkan, tetapi dicatat flag overbudget untuk review auditor.
- **Require Approval:** transaksi di-hold hingga Budget Owner level lebih tinggi atau Finance Manager approve.

Policy dapat dikonfigurasi per budget component. Misalnya, component "Gaji" selalu Block (tidak boleh overbudget), sedangkan component "Marketing" bisa Require Approval dengan threshold tertentu. Konfigurasi ini disimpan di tabel `budget_component.policy` sebagai JSON.

Laporan Variance

Setiap akhir periode, modul Budgeting menghasilkan **variance report** yang membandingkan anggaran vs realisasi per departemen dan komponen. Report menampilkan kolom: Budget, Actual, Encumbrance (PO yang belum invoiced), Available, Variance (Rp), Variance (%). Variance negatif (overbudget) disorot dengan warna merah, variance positif (underbudget) dengan hijau. Report dapat di-drill-down hingga ke transaksi individual yang menyebabkan variance.

SECTION 6

Pemantauan Transaksi

Tujuan Modul

Modul **Pemantauan Transaksi** memberikan visibilitas real-time terhadap seluruh transaksi yang terjadi di sistem GIL. Modul ini menjawab kebutuhan klasik finance manager: "Apa yang sedang terjadi hari ini?" tanpa harus menunggu laporan akhir bulan. Dashboard menampilkan stream transaksi terbaru, agregasi per kategori, anomaly flag, dan status approval. Audit trail yang komprehensif menjadi sumber data utama untuk forensik dan investigasi.

Komponen Utama

Komponen	Fungsi
Real-time Dashboard	Stream transaksi terbaru (WebSocket), agregasi hari ini (count, value, top 10), alert banner.
Audit Trail	Log setiap aksi (create, update, delete, approve, reject, view-sensitive) dengan user, timestamp, IP, before/after JSON.
Anomaly Detection	Rule-based: transaksi besar (>% threshold), transaksi di luar jam kerja, duplikasi nominal, akun-akun sensitif.
Approval Workflow Status	Daftar dokumen pending approval per user, dengan SLA dan eskalasi.
Alert & Notification	Push notification in-app, email, opsional Slack/Telegram webhook.
Filter Multi-Dimensi	Filter by date, user, department, module, amount range, status, anomaly flag.
Export	Export CSV/Excel/PDF untuk audit forensik, dengan digital signature untuk keaslian.

Audit Trail Detail

Setiap aksi yang mengubah state data di GIL tercatat di tabel `audit_logs`. Struktur log mencakup: **id** (UUID), **timestamp** (with timezone), **user_id**, **action** (create/update/delete/approve/reject), **entity_type** (tabel yang diakses), **entity_id**, **before** (JSON snapshot sebelum perubahan), **after** (JSON snapshot setelah), **ip_address**, **user_agent**, dan **request_id** (untuk korelasi). Log bersifat append-only — tidak ada UPDATE atau DELETE pada tabel ini, bahkan oleh Super Admin.

Untuk performa, audit log di-partisi per bulan pada PostgreSQL. Query untuk audit forensik biasanya berbasis rentang waktu, sehingga partisi bulanan memungkinkan PostgreSQL memilih partition yang relevan saja. Retention default adalah 7 tahun (sesuai regulasi pajak Indonesia), setelah itu data diarsipkan ke cold storage (S3/Glacier) dalam format Parquet.

Anomaly Detection

Sistem anomaly detection GIL bersifat **rule-based** pada MVP, dengan rencana migrasi ke **ML-based** pada fase berikutnya. Rule yang tersedia pada MVP:

- **Large Amount:** transaksi dengan nilai $> N\%$ dari rata-rata transaksi jenis sama (default 3x std dev).
- **Off-Hours Activity:** transaksi yang dibuat di luar jam kerja default (Senin-Jumat 07:00-19:00).
- **Duplicate Amount:** transaksi dengan nominal sama pada hari yang sama oleh user yang sama.
- **Sensitive Account:** akses atau perubahan pada akun sensitif (misal: akun ekuitas, akun pajak).
- **Rapid Sequence:** lebih dari X transaksi oleh user yang sama dalam Y menit.

Setiap anomali yang terdeteksi diberi flag pada kolom `transactions.anomaly_flags` (array JSON) dan muncul di dashboard dengan highlight visual. Auditor dapat menandai anomali sebagai "reviewed — false positive" atau "escalated" untuk follow-up lebih lanjut.

SECTION 7

Close/Open Period

Konsep Periode

Modul **Close/Open Period** memberikan kontrol ketat terhadap periode akuntansi. Setiap transaksi di GIL terikat pada sebuah periode (default: bulanan, dapat dikonfigurasi ke quarterly atau custom). Periode memiliki state yang menentukan apakah transaksi baru dapat diposting, dan apakah transaksi existing masih dapat diedit. Mekanisme ini menjamin integritas pelaporan keuangan: setelah laporan bulan Januari disahkan, tidak ada transaksi Januari yang dapat disisipkan diam-diam.

State Periode

State	Definisi	Transaksi Baru	Edit Transaksi Existing
Open	Periode aktif, transaksi normal	Diizinkan (sesuai RBAC)	Diizinkan (jika jurnal belum posted)
Soft-Close	Marked closed, masih bisa reopen	Ditolak (kecuali override)	Ditolak (perlu reopen dulu)
Hard-Close	Final, reopen sangat terbatas	Ditolak mutlak	Ditolak mutlak
Reopened	Sementara dibuka kembali dari Soft-Close	Diizinkan dengan audit flag	Diizinkan dengan audit flag

Checklist Close Period

Sebelum sebuah periode dapat di-soft-close, sistem menjalankan **pre-close checklist** otomatis. Checklist ini memastikan tidak ada transaksi yang tertinggal atau proses yang belum selesai. Jika ada item yang gagal, close ditolak dan finance manager harus menyelesaikan terlebih dahulu. Berikut checklist default (dapat dikustomisasi per organisasi):

- Semua jurnal draft sudah diposting atau dihapus.
- Tidak ada dokumen (PO, SO, Invoice) yang berstatus pending approval.
- Rekonsiliasi bank untuk semua akun bank periode ini sudah completed.
- Penyusutan aset tetap untuk periode ini sudah dijalankan.
- Stock opname (jika ada) sudah final dan variance-nya sudah dijurnal.
- Variance report budgeting sudah di-generate dan di-review oleh finance manager.
- Pre-close trial balance sudah balanced (total debit = total kredit).

Reopen Period dengan Audit

Soft-close dapat di-reopen oleh Finance Manager atau Super Admin dengan alasan yang dicatat. Setiap reopen menghasilkan entri pada **period_reopen_logs** yang menyimpan: **period_id**, **user_id**, **timestamp**, **reason**, list of transactions yang akan ditambahkan/diedit, dan approver

(untuk dua-level approval). Reopen otomatis mengembalikan state periode ke **Reopened**, dan setiap transaksi baru yang diposting akan diberi flag `is_reopened_period=true` untuk transparansi pelaporan.

Hard-close bersifat final dan hanya dapat di-reopen melalui **audit-approved reversal**: permintaan diajukan, disetujui oleh auditor eksternal, dan ditandatangani secara digital oleh Finance Manager + Direktur. Hal ini sesuai prinsip "auditability first" — sekali laporan tahunan disahkan, koreksi harus melalui mekanisme formal, bukan edit diam-diam.

SECTION 8

Role-Based Access Control (RBAC)

Konsep RBAC

GIL mengimplementasikan RBAC dengan model **user** → **role** → **permission** yang klasik, dilengkapi dengan dimensi **department scope** untuk kontrol yang lebih granular. Seorang user dapat memiliki multiple roles, dan permission efektif adalah union dari semua permission pada role-role tersebut. Department scope memungkinkan role yang sama memiliki cakupan data berbeda — misalnya, seorang "Accountant" dengan scope departemen IT hanya bisa melihat dan memposting transaksi departemen IT, tidak bisa menyentuh data departemen HR.

Entitas RBAC

Entitas	Field Kunci	Deskripsi
User	id, email, name, is_active, department_id	Akun login; terikat ke satu departemen utama.
Role	id, code, name, description, is_system	Kumpulan permission; system role tidak dapat dihapus.
Permission	id, code, module, action, description	Hak aksi atomik: module.action (e.g. budget.create).
RolePermission	role_id, permission_id	Many-to-many antara role dan permission.
UserRole	user_id, role_id, department_scope	Role yang dimiliki user, dengan scope departemen opsional.
Department	id, code, name, parent_id	Hierarki departemen untuk scope.

Matrix Role-Permission Default

Berikut adalah role default GIL beserta permission utamanya. Super Admin dapat membuat role custom dan mengkustomisasi matrix ini, namun role system (Super Admin, Finance Manager, Auditor) tidak dapat dihapus dan permission intinya tidak dapat dicabut.

Role	GL	AR/AP	Sales/Purchase	Inventory	Budget	Period Close	RBAC	Audit Trail
Super Admin	Full	Full	Full	Full	Full	Full	Full	Full
Finance Manager	Full	Full	Approve	View	Approve	Soft+Hard	View	Full
Accountant	CRUD	CRUD	CRUD	View	View	—	—	View own
Budget Owner	View	View	View	View	CRUD own dept	—	—	View own dept
Department Head	View	View	View	View	Approve own dept	—	—	View own dept
Auditor	View	View	View	View	View	View	View	Full read
Staff (Purchaser)	—	View	Create PO	View	View own	—	—	—

Role	GL	AR/AP	Sales/Purchase	Inventory	Budget	Period Close	RBAC	Audit Trail
Staff (Sales)	—	View	Create SO	View	View own	—	—	—

Keterangan: Full = semua aksi; CRUD = Create/Read/Update/Delete; View = read-only; Approve = approval workflow; — = tidak ada akses; "own dept" = terbatas pada departemen user.

Implementasi Backend

Pada backend Nuxt (Nitro server), otorisasi diimplementasikan melalui **middleware** yang berjalan sebelum handler endpoint. Middleware ini membaca user dari session, mengambil role + permission, lalu memeriksa apakah permission yang diminta (e.g. `budget.create`) ada dalam set permission user. Jika tidak, response 403 dikembalikan. Untuk department scope, middleware menambahkan filter `WHERE department_id = user.department_id` pada query data.

```
// server/middleware/authz.ts
export default defineEventHandler(async (event) => {
  const user = event.context.user
  if (!user) return
  const perms = await getUserPermissions(user.id)
  const required = getRequiredPermission(event)
  if (required && !perms.has(required)) {
    throw createError({ statusCode: 403, message: "Forbidden" })
  }
})
```

Implementasi Frontend

Pada frontend Nuxt (Vue 3), RBAC diimplementasikan melalui **route middleware** dan **composable usePermission**. Route middleware memeriksa apakah user memiliki permission untuk mengakses halaman tersebut (misal: halaman `/budget/create` memerlukan `budget.create`). Composable `usePermission` digunakan di tingkat komponen untuk menyembunyikan tombol, menu, atau field yang tidak boleh diakses user. Catatan: kontrol frontend hanya bersifat UX — keamanan sebenarnya tetap di backend.

SECTION 9

Model Data & Skema Database

Tabel Utama

Berikut adalah tabel utama pada skema PostgreSQL GIL, dikelompokkan per domain. Untuk keringkasan, tabel hanya menampilkan field kunci; field audit standar (`created_at`, `updated_at`, `created_by`, `updated_by`) ada di semua tabel dan tidak ditampilkan.

Domain	Tabel	Field Kunci
Auth & RBAC	users	id, email, password_hash, name, department_id, is_active
	roles	id, code, name, is_system
	permissions	id, code, module, action
	role_permissions	role_id, permission_id
	user_roles	user_id, role_id, department_scope_id
	departments	id, code, name, parent_id, head_user_id
GL	accounts	id, code, name, type, parent_id, department_id, currency
	fiscal_periods	id, name, start_date, end_date, status, fiscal_year_id
	journals	id, number, date, period_id, description, status
	journal_lines	id, journal_id, account_id, debit, credit, department_id, project_id
AR/AP	customers	id, code, name, credit_limit, term_id, ar_account_id
	suppliers	id, code, name, term_id, ap_account_id
	ar_invoices	id, number, customer_id, date, due_date, amount, status
	ap_invoices	id, number, supplier_id, date, due_date, amount, status
	receipts	id, number, customer_id, date, amount, method, bank_account_id
	payments	id, number, supplier_id, date, amount, method, bank_account_id
Inventory	items	id, code, name, type, unit, valuation_method, default_account_id
	warehouses	id, code, name, location
	stock_cards	id, item_id, warehouse_id, date, qty_in, qty_out, balance, ref_doc_id
	boms	id, finished_item_id, version, status
	bom_lines	id, bom_id, component_item_id, qty, unit
Budgeting	budget_components	id, code, name, type, account_id, policy_json
	budgets	id, department_id, component_id, period_id, amount, status
	budget_lines	id, budget_id, transaction_line_id, amount, type (encumbrance/actual)
	budget_transfers	id, from_budget_id, to_budget_id, amount, approver_id, reason

Domain	Tabel	Field Kunci
Operations	transactions	id, type, number, date, department_id, status, anomaly_flags
	transaction_lines	id, transaction_id, account_id, debit, credit, budget_id
	audit_logs	id, timestamp, user_id, action, entity_type, entity_id, before_json, after_json
	period_reopen_logs	id, period_id, user_id, reason, approver_id, timestamp

Contoh DDL — Tabel Budgets

```
CREATE TABLE budgets (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  department_id UUID NOT NULL REFERENCES departments(id),
  component_id UUID NOT NULL REFERENCES budget_components(id),
  period_id UUID NOT NULL REFERENCES fiscal_periods(id),
  amount NUMERIC(18,2) NOT NULL CHECK (amount >= 0),
  status VARCHAR(20) NOT NULL DEFAULT 'draft'
  CHECK (status IN ('draft','review','approved','active','closed')),
  approved_by UUID REFERENCES users(id),
  approved_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  created_by UUID REFERENCES users(id),
  updated_by UUID REFERENCES users(id),
  UNIQUE (department_id, component_id, period_id)
);

CREATE INDEX idx_budgets_dept ON budgets(department_id);
CREATE INDEX idx_budgets_period ON budgets(period_id);
CREATE INDEX idx_budgets_status ON budgets(status);
```

SECTION 10

Tech Stack & Konfigurasi

Stack Utama

Lapisan	Teknologi	Justifikasi
Frontend	Nuxt 3 + Vue 3 (Composition API)	SSR/SPA hybrid, file-based routing, auto code-splitting.
UI Framework	Tailwind CSS + shadcn-vue	Utility-first, konsistensi desain, komponen accessible.
State Management	Pinia	Type-safe, devtools, official Vue recommendation.
Backend	Nuxt 3 server routes (Nitro)	Satu codebase dengan frontend, TypeScript end-to-end.
ORM	Drizzle ORM	Type-safe, SQL-first, schema migration, performa tinggi.
Database	PostgreSQL 16	ACID, JSONB, partitioning, row-level security, mature ecosystem.
Cache & Session	Redis 7	Session store, cache query berat, rate limiting, pub/sub untuk WebSocket.
Validation	Zod	Schema validation shared antara client & server.
Auth	nuxt-auth + JWT cookie httpOnly	CSRF protection, session expiry, refresh token rotation.
Background Job	BullMQ (Node) / pg-boss	Async task: email, laporan terjadwal, retry.
Search	PostgreSQL FTS / Meilisearch (opsional)	Full-text search untuk audit log & dokumen.
Testing	Vitest + Playwright	Unit/integration (Vitest), E2E (Playwright).
CI/CD	GitHub Actions + Docker	Build, test, scan, deploy otomatis.
Monitoring	Sentry + OpenTelemetry + Grafana	Error tracking, tracing, dashboard operasional.

Konfigurasi Project

Project structure mengikuti konvensi Nuxt 3 dengan pemisahan yang jelas antara modul domain. Setiap modul memiliki folder sendiri di bawah `server/api/` untuk endpoint, `server/service/` untuk business logic, dan `server/db/schema/` untuk definisi skema Drizzle. Frontend menggunakan struktur `pages/` + `components/` standar Nuxt, dengan Pinia store per modul.

```
gil/
├─ nuxt.config.ts
├─ package.json
├─ drizzle.config.ts
├─ server/
│  ├─ api/[module]/  # REST endpoints
│  └─ service/[module]/  # Business logic
```

```
| ├── middleware/ # auth, rbac, rate-limit  
| ├── db/schema/ # Drizzle schemas per module  
| └── utils/ # Shared utilities  
├── pages/  
| ├── index.vue  
| ├── budget/  
| ├── ledger/  
| └── ...  
├── components/  
├── stores/ # Pinia stores  
├── composables/  
└── assets/
```

Deployment & DevOps

Untuk on-premise deployment, GIL didistribusikan sebagai **Docker Compose bundle**: file `docker-compose.yml` + `.env.example` + dokumen setup. Customer menjalankan `docker compose up -d` untuk menjalankan aplikasi. Untuk cloud/SaaS deployment, GIL menggunakan Kubernetes dengan Helm chart, autoscaling berbasis CPU/RAM, dan PostgreSQL managed (RDS/Cloud SQL) untuk high availability. Backup database dijalankan otomatis setiap 6 jam dengan retention 30 hari, plus full backup mingguan dengan retention 7 tahun untuk kepatuhan pajak.

SECTION II

Roadmap Implementasi

GIL dikembangkan secara iteratif dalam empat fase, masing-masing menghasilkan release yang dapat di-deploy dan diuji oleh pilot customer. Pendekatan ini memungkinkan tim mendapatkan feedback awal sebelum seluruh modul selesai, serta memitigasi risiko dengan validasi asumsi secara bertahap. Estimasi timeline di bawah mengasumsikan tim inti 4-6 engineer (1 tech lead, 2-3 fullstack, 1 QA, opsional 1 designer).

Fase	Durasi	Modul	Deliverable Utama
Fase 1 MVP	Bulan 1-4	Auth, RBAC, GL, Master Data	User login, role-permission matrix, chart of account, jurnal umum, trial balance. Pilot internal.
Fase 2 Core Ops	Bulan 5-9	AR, AP, Sales, Purchase, Inventory, Cash & Bank	Siklus penjualan/pembelian end-to-end, faktur, penerimaan/pembayaran, kartu stok, rekonsiliasi bank.
Fase 3 Budgeting & Monitoring	Bulan 10-14	Budgeting, Transaction Monitoring, Fixed Asset, Manufacturing	Budget per departemen, kontrol overbudget, dashboard real-time, audit trail, penyusutan aset, work order.
Fase 4 Governance & Scale	Bulan 15-18	Close/Open Period, Advanced Reporting, Multi-currency, Integrasi	Soft/hard close, reopen audit, e-Faktur, e-Bupot, multi-currency, dashboard executive, API publik.

Milestone & Success Criteria

- **End Fase 1 (Bulan 4):** MVP dapat menjalankan siklus akuntansi dasar (jurnal → trial balance → laporan keuangan). Pilot internal berjalan 1 bulan tanpa data corruption.
- **End Fase 2 (Bulan 9):** 3 pilot customer eksternal onboard. Siklus penjualan-pembelian-persediaan berjalan end-to-end di production.
- **End Fase 3 (Bulan 14):** Modul budgeting dipakai pilot customer untuk periode budgeting Q1 berikutnya. Audit trail lulus review auditor eksternal.
- **End Fase 4 (Bulan 18):** General availability. 20+ tenant aktif. Sertifikasi kepatuhan PSAK & e-Faktur.

Risiko & Mitigasi

Risiko	Likelihood	Impact	Mitigasi
Scope creep modul	Tinggi	Tinggi	Roadmap lock per fase; perubahan via change request formal.
Kompleksitas migrasi data dari Accurate	Sedang	Tinggi	Tool migrasi dibangun di Fase 2; pilot customer dipilih yang willing migrate.
Kinerja dashboard real-time	Sedang	Sedang	Redis cache, materialized view, WebSocket dengan batching.

Risiko	Likelihood	Impact	Mitigasi
Perubahan regulasi pajak	Tinggi	Sedang	Modul pajak modular, update tanpa refactor besar.
Adopsi user (training gap)	Sedang	Tinggi	UI/UX invest, video tutorial, role-based onboarding.

SECTION 12

Penutup

GIL dirancang sebagai **evolusi** dari kelas aplikasi akuntansi yang diwakili Accurate, bukan sekadar replikasi. Dengan menambahkan empat modul khas enterprise — **Budgeting per Departemen, Pemantauan Transaksi, Close/Open Period**, dan **RBAC granular** — GIL menjawab kebutuhan organisasi modern yang menuntut kontrol anggaran terdistribusi, visibilitas real-time, integritas periode, dan tata kelola akses yang ketat. Pilihan stack Nuxt 3 + PostgreSQL memberikan foundation modern, type-safe, dan mudah di-deploy tanpa vendor lock-in.

Bagi tim engineering, dokumen ini menjadi rujukan untuk memulai implementasi Fase 1. Langkah berikutnya adalah menyusun **engineering kickoff**: setup repository, konvensi code review, CI/CD pipeline, lalu mulai spike untuk modul Auth + RBAC + GL. Bagi stakeholder product, dokumen ini menjadi dasar untuk validasi prioritas modul dengan pilot customer potensial. Bagi tim bisnis, dokumen ini dapat menjadi bahan briefing untuk calon investor atau partner strategis.

Filosofi nama GIL — mata uang universal yang melintasi kerajaan dan era — mengingatkan kita bahwa platform yang baik harus tahan waktu, lintas use case, dan menjadi fondasi yang dipercaya oleh banyak pihak. Mari bangun GIL dengan standar tertinggi: auditability, integrity, discipline, least privilege. Selamat berkarya.